

[CC3067] Redes

Proyecto 1

Implementación de un protocolo existente

Nombre	Carné
Daniel Chet	231177

0. Resumen

Este informe documenta la implementación completa de un chatbot de terminal que consume un modelo de lenguaje y coordina servidores Model Context Protocol (MCP). El resultado final cumple el flujo solicitado: conserva contexto conversacional, registra todas las interacciones MCP, integra los servidores oficiales Filesystem y Git, ofrece un servidor industrial propio en modo local y remoto, despliega ese servidor en Google Cloud Run y analiza una sesión real mediante Wireshark.

El servidor desarrollado asiste consultas sobre compresores KAESER mediante cinco herramientas: especificaciones, diagnóstico de mensajes, mantenimiento por horas, disponibilidad de repuestos y registro de sensores. MCP y JSON-RPC 2.0 se implementaron manualmente en Node.js, sin SDK de MCP. La demostración final utilizó Gemini 3.5 Flash Lite, descubrió 31 herramientas y ejecutó solicitudes reales contra Cloud Run.

Resultados	Evidencia
Servidor MCP propio	5 herramientas disponibles por stdio y HTTP
Integración completa	5 KAESER + 14 Filesystem + 12 Git = 31 herramientas
Despliegue	Servicio kaeser-mcp activo en Google Cloud Run
Pruebas	6/6 pruebas del servidor y 11/11 pruebas del chatbot
Filesystem + Git	Commit fbe71af creado dentro de demo-workspace
Captura de red	DNS, TCP, TLS 1.3, HTTP/1.1 y JSON-RPC correlacionados

Alcance y advertencia

El proyecto es un prototipo de demostración. No controla maquinaria física y no sustituye manuales, procedimientos de planta, bloqueo y etiquetado (LOTO), equipo de protección personal ni servicio técnico autorizado. El historial de mantenimiento, inventario, números de serie, rangos y lecturas de sensores son datos simulados.

1. Requisitos y trazabilidad

La siguiente matriz muestra los requisitos del proyecto y la evidencia disponible.

No.	Requisito	Implementación
1	Conectar un LLM	Cliente REST para Gemini; Groq y Claude quedan como alternativas.
2	Conservar contexto	Historial de usuario, asistente, llamadas y resultados durante la sesión.
3	Mostrar interacciones MCP	Log JSONL con mensajes enviados/recibidos y comando /log.
4	Filesystem + Git	Escenario real aislado en demo-workspace con escritura, add, commit y log.
5	Servidor propio local	Servidor KAESER manual por stdio, documentación, instalación y ejemplos.
6	Servidor propio remoto	Misma lógica publicada en Cloud Run mediante Streamable HTTP.
7	Wireshark	Captura real descifrada y correlación de sincronización, solicitudes y respuestas.
8	Especificación	Métodos, parámetros, esquemas de salida, errores y endpoints documentados.
9	Capas de red	Análisis de enlace, red, transporte y aplicación con valores observados.
10	Conclusiones	Resultados, dificultades, aprendizajes, seguridad y limitaciones.

2. Arquitectura de la solución

El chatbot actúa como anfitrión MCP. Descubre las herramientas de cada servidor, adapta sus esquemas al formato de funciones del LLM, ejecuta la herramienta solicitada y devuelve el resultado al modelo. La lógica industrial es la misma en los transportes local y remoto.

Usuario	Anfitrión	Integraciones	Servidores
---------	-----------	---------------	------------

Terminal	Chatbot Node.js Contexto + auditoría	Gemini REST Clientes MCP	KAESER local/remoto Filesystem Git
----------	---	-----------------------------	--

- **Anfitrión:** interfaz de terminal, selección del proveedor, ciclo de herramientas y comandos de operación.
- **Cliente MCP stdio:** JSON-RPC delimitado por líneas para procesos locales.
- **Cliente MCP HTTP:** solicitudes POST /mcp con respuestas JSON y autenticación Bearer.
- **Servidor KAESER:** validación, enrutamiento, esquemas y reglas industriales compartidas.
- **Auditoría:** un archivo JSONL por sesión con tiempo UTC, servidor, transporte, dirección y contenido MCP.

2.1 Flujo de una consulta

1. El anfitrión inicializa cada servidor y solicita tools/list.
2. Las herramientas descubiertas se entregan al LLM como declaraciones de funciones.
3. Gemini devuelve una llamada de función con nombre y argumentos estructurados.
4. El anfitrión envía tools/call al servidor MCP correspondiente.
5. El resultado MCP vuelve al modelo como functionResponse y se incorpora al historial.
6. La respuesta final se muestra en la terminal y las interacciones quedan en el log.

3. Chatbot y modelo de lenguaje

3.1 API de Gemini

La demostración final usó el proveedor gemini y el modelo gemini-3.5-flash-lite. El cliente invoca por HTTPS el método generateContent, envía el historial y las declaraciones de funciones, y autentica con una clave almacenada únicamente en GEMINI_API_KEY o GOOGLE_API_KEY. Ninguna clave se almacena en archivos versionados.

El adaptador conserva los identificadores de llamadas, transforma el resultado MCP en functionResponse y preserva las firmas de pensamiento que devuelve la API. También reintenta con espera exponencial errores 408, 429, 5xx, timeouts y fallos temporales de red.

3.2 Contexto conversacional

El historial contiene turnos del usuario, respuestas del asistente, llamadas y resultados de herramientas. Así, una pregunta posterior puede depender de información previa. El comando /clear permite comenzar una conversación nueva sin reiniciar los servidores.

3.3 Comandos y observabilidad

Comando	Función
/help	Muestra la ayuda del chatbot.
/tools	Lista todas las herramientas descubiertas.
/log n	Muestra las últimas n entradas del log MCP.

/clear	Limpia el contexto de conversación.
/exit	Cierra conexiones y procesos MCP.

```
npm run start

= kaeser-mcp-chatbot@0.0.0 start
= node chatbot.js

KAESER MCP Terminal ChatBot
Project: C:\VWD\Semestra8\CC3867 - Redes\Proyectos\Repo\CC3867-Redes
LLM provider: gemini
Model: gemini-1.5-flash-lite
MCP log: C:\VWD\Semestra8\CC3867 - Redes\Proyectos\Repo\CC3867-Redes\chatbot\logs\mcp-2426-89-09716-11-18-1847.json

[MCP] Connecting to kaeser_remote using http...
[MCP] kaeser_remote connected (15 tools)
[MCP] Connecting to filesystem using stdio...
[MCP] filesystem connected (24 tools)
[MCP] Connecting to git using stdio...
[MCP] git connected (12 tools)

Ready with 31 MCP tools. Type /help for commands.

You: Cuales son las especificaciones del SM 13.
[MCP] Calling kaeser_remote/consultar_specs
{
  "modelo": "SM 13"
}

Gemini: A continuación se presentan las especificaciones técnicas oficiales y simuladas para el modelo de compresor "SM 13", obtenidas a través de la herramienta de consulta de especificaciones técnicas:

+ **Familia:** SM
+ **Configuración:** Standard
+ **Potencia:** 7.5 kW
+ **Caudal:** 1.32 m³/min
+ **Presión de trabajo:** 7.5 bar
+ **Presión máxima:** 9 bar
+ **Dimensiones (Ancho x Fondo x Alto):** 638 x 798 x 1186 mm
+ **Fuente:** Folleto de la serie KAESER SM, especificaciones técnicas.

...

> ⚠️ Advertencia de seguridad y uso industrial: ⚠️
> Esta información es una simulación/prototipo de demostración y no sustituye el manual oficial del equipo ni los procedimientos de seguridad de planta establecidos.
> Asegúrese de controlar minuciosamente todos los datos remota a las debidas certificaciones, bloques (IOT) y protocolos de seguridad industriales.
```

Figura 1. Chatbot conectado a 31 herramientas y ejecución remota de consultar_specs con Gemini 3.5 Flash Lite.

4. Servidores MCP oficiales

Filesystem se ejecuta mediante @modelcontextprotocol/server-filesystem y recibe acceso exclusivo a demo-workspace. Git se ejecuta mediante uvx mcp-server-git sobre el mismo repositorio aislado. Esta separación evita que una demostración del LLM altere el repositorio principal.

Servidor	Transporte	Herramientas	Restricción
Filesystem	stdio	14	Solo demo-workspace
Git	stdio	12	repo_path fijo en demo-workspace

4.1 Escenario ejecutado

La prueba real creó README.md con filesystem/write_file, consultó git_status, preparó el archivo con git_add, creó un commit con git_commit y confirmó el historial con git_log. El segundo turno del usuario continuó el trabajo pendiente, demostrando además la conservación del contexto.

Dato	Valor observado
Repositorio	demo-workspace
Archivo	README.md
Commit	fbe71af49cfe51314ecebc996ebefeab7a5f9985

Mensaje	Add initial README to demo-workspace
Estado final	Árbol de trabajo limpio

```

// MCP Start
// MCP Tools, Type /help for commands.

// Trabaja únicamente dentro del directorio autorizado demo-workspace.

// Usa obligatoriamente las herramientas MCP de Filesystem y Git para realizar estas acciones en orden:
1. Usa filesystem_write_file para crear README.md dentro de demo-workspace.
2. El README debe titularse "KAESER Maintenance Demo" y explicar brevemente que es un repositorio de demostración para mantenimiento preventivo de computadores.
3. Usa git_status para comprobar el estado del repositorio.
4. Usa git_add para agregar README.md.
5. Usa git_commit para crear el commit con el mensaje "docs: add demo readme".
6. Usa git_log para mostrar el commit creado.

// Para todas las operaciones git utilice como repo_path:
C:\Users\SemestreB\CC3607 - Redes\Proyectos\Repos\CC3607-Redes\demo-workspace

// No utilice el repositorio principal y no se limite a explicar los cambios: ejecuta las herramientas MCP.

// Señal: Entendido. Trabaja exclusivamente dentro del directorio autorizado "demo-workspace" utilizando las herramientas disponibles y siguiendo todas las directrices de seguridad e integridad técnica establecidas. ¿En qué te ayudo ahora?

// Continúa con los pasos pendientes. Debes ejecutar git_add, git_commit y git_log mediante las herramientas MCP.
[{"tool": "git_status",
  "args": {
    "repo_path": "C:\\Users\\SemestreB\\CC3607 - Redes\\Proyectos\\Repos\\CC3607-Redes\\demo-workspace"
  }
}, {"tool": "filesystem_write_file",
  "args": {
    "path": "C:\\Users\\SemestreB\\CC3607 - Redes\\Proyectos\\Repos\\CC3607-Redes\\demo-workspace",
    "pattern": "README.md",
    "content": "KAESER Maintenance Demo\n\nEste repositorio de demostración es para mantenimiento preventivo de computadores."
  }
}, {"tool": "git_add",
  "args": {
    "repo_path": "C:\\Users\\SemestreB\\CC3607 - Redes\\Proyectos\\Repos\\CC3607-Redes\\demo-workspace"
  }
}, {"tool": "git_commit",
  "args": {
    "repo_path": "C:\\Users\\SemestreB\\CC3607 - Redes\\Proyectos\\Repos\\CC3607-Redes\\demo-workspace",
    "message": "docs: add demo readme"
  }
}, {"tool": "git_log",
  "args": {
    "repo_path": "C:\\Users\\SemestreB\\CC3607 - Redes\\Proyectos\\Repos\\CC3607-Redes\\demo-workspace"
  }
}]

// Señal: Se ejecutaron exitosamente los pasos pendientes utilizando las herramientas MCP dentro del directorio autorizado "demo-workspace".

1. Se creó un archivo README.md de prueba dentro de "demo-workspace".
2. Se agregó el archivo al área de preparación (staging area).
3. Se realizó el commit con el mensaje "docs: add demo readme".
4. Se consultó el historial de commits para verificar el registro de la operación.
```

Figura 2. Ejecución de las herramientas oficiales Filesystem y Git y confirmación del commit aislado.

5. Servidor MCP KAESER

5.1 Métodos del protocolo

Método	Clase	Propósito
initialize	Solicitud	Negocia versión, identidad y capacidades.
notifications/initialized	Notificación	Confirma que la inicialización terminó.
ping	Solicitud	Comprueba disponibilidad del servidor.
tools/list	Solicitud	Devuelve las cinco herramientas y sus esquemas.
tools/call	Solicitud	Ejecuta una herramienta con argumentos validados.

5.2 Herramientas y parámetros

Herramienta	Parámetros requeridos	Salida principal
-------------	-----------------------	------------------

consultar_specs	modelo: string	Familia, configuración, presión, caudal, potencia y dimensiones.
diagnosticar_falla	codigo_mensaje: string; modelo: string	Tipo, descripción, causas, acciones y necesidad de servicio.
calcular_mantenimiento_pendiente	numero_serie: string; horas_operacion: number ≥ 0	Estado, servicio anterior/siguiente y horas restantes o vencidas.
consultar_disponibilidad_repuesto	numero_parte: string	Cantidad, bodega, reabastecimiento y actualización.
registrar_lectura_sensor	numero_serie; tipo; valor; unidad	Timestamp, rango configurado, nivel de alerta y registro.

Cada salida exitosa incluye fuente y advertencia. Los argumentos inválidos o datos ausentes se representan como resultados MCP con `isError: true`; los problemas del protocolo utilizan el objeto error de JSON-RPC 2.0. Esta separación permite distinguir un fallo de negocio de un fallo de transporte.

5.3 Endpoints remotos

Método y ruta	Autenticación	Comportamiento
GET /health	No	Estado básico del servicio.
POST /mcp	Bearer	JSON-RPC; HTTP 202 para notificaciones sin respuesta.
OPTIONS /mcp	No	Preflight CORS.
GET /mcp	Bearer	HTTP 405; no hay stream SSE independiente.
DELETE /mcp	Bearer	HTTP 405; el servidor no mantiene sesiones.

El endpoint valida Content-Type, Accept, MCP-Protocol-Version, tamaño máximo, token y Origin. La implementación utiliza el modo Streamable HTTP con respuestas JSON y sin estado.

6. Ejecución local y despliegue en nube

6.1 Transporte local

El proceso local lee JSON-RPC desde stdin y escribe exclusivamente mensajes del protocolo en stdout. Un buffer por líneas admite mensajes fragmentados o múltiples mensajes por lectura. Los diagnósticos se envían a stderr para no corromper el canal.

6.2 Google Cloud Run

El contenedor usa Node.js 22 Alpine, instala dependencias reproducibles mediante npm ci, ejecuta como usuario sin privilegios y escucha en 0.0.0.0:\$PORT. Cloud Run termina TLS y reenvía la petición al contenedor.

Elemento	Valor
Proyecto	cc3067-proyecto1
Región	us-central1
Servicio	kaeser-mcp
Revisión	kaeser-mcp-00002-g27
URL	https://kaeser-mcp-690519080181.us-central1.run.app
Endpoint	https://kaeser-mcp-690519080181.us-central1.run.app/mcp
Validación	9 de septiembre de 2026

En local, registrar_lectura_sensor persiste en data/lecturas.json. En Cloud Run utiliza /tmp/kaeser-lecturas.json, almacenamiento efímero apropiado únicamente para demostración. Una versión productiva requeriría una base de datos durable.

7. Resultados funcionales

Consulta	Herramienta	Resultado
Especificaciones SM 13	consultar_specs	7.5 kW; 1.32 m³/min; 7.5 bar de trabajo; 8 bar máximo; 630 × 790 × 1100 mm.
Código 0044 A	diagnosticar_falla	Alarma No pressure buildup; verificar fugas, acoplamiento, correa y válvulas; requiere servicio autorizado.
KC-2291 con 5900 h	calcular_mantenimiento_pendiente	Estado próximo; servicio a 6000 h; restan 100 h.
Número de parte inexistente	consultar_disponibilidad_repuesto	Resultado MCP isError; el transporte HTTP continuó siendo 200.

Estas respuestas se obtuvieron mediante el servidor remoto de Cloud Run. La salida final del LLM añadió contexto legible y mantuvo la advertencia industrial definida por el servidor.

8. Análisis de comunicación con Wireshark

La captura real permitió relacionar el protocolo de aplicación con las capas inferiores. Se exportaron temporalmente los secretos TLS de una sesión propia para que Wireshark pudiera mostrar HTTP/1.1 y el JSON transportado por POST /mcp. Ninguna clave de API ni token Bearer se incluye en este informe.

Propiedad de captura	Valor
Interfaz	Intel(R) Wi-Fi 6E AX211 160MHz
Encapsulación	Ethernet
Duración	199.7188402 s
Paquetes	Aproximadamente 165 000
Cliente	10.100.3.195:63207
Servidor observado	34.143.74.2:443
Nombre SNI	kaeser-mcp-690519080181.us-central1.run.app

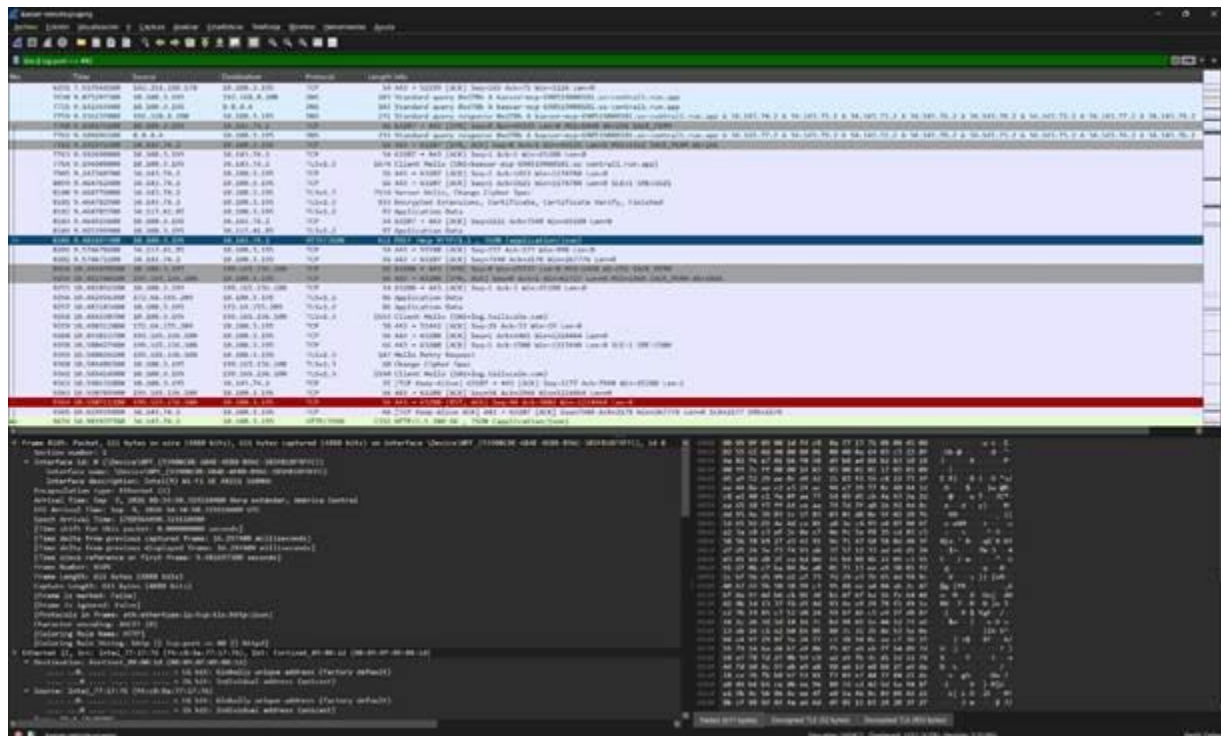


Figura 3. Vista general de DNS, establecimiento TCP, negociación TLS y mensajes HTTP de la sesión remota.

8.1 Capa de enlace

Npcap entregó a Wireshark tramas con encapsulación Ethernet. En la trama 8105 se observó la MAC origen f4:c8:8a:77:17:76, la MAC destino 00:09:0f:09:00:1d y EtherType

0x0800 para IPv4. La MAC destino corresponde al gateway local Fortinet, no a Cloud Run, porque las direcciones MAC solo tienen alcance en el enlace local.

8.2 Capa de red

IPv4 proporcionó direccionamiento y enrutamiento entre 10.100.3.195 y el frontend de Google 34.143.74.2. La solicitud de la trama 8105 salió con TTL 128 y la respuesta de la trama 9674 llegó con TTL 121. No se observó fragmentación en el flujo. La dirección pública puede cambiar porque Cloud Run utiliza infraestructura frontal compartida.

8.3 Capa de transporte

El cliente abrió el puerto efímero 63207 hacia TCP 443. El three-way handshake aparece en las tramas 7760 (SYN), 7762 (SYN/ACK) y 7763 (ACK). La conexión persistente se reutilizó para todas las solicitudes MCP y Wireshark no marcó retransmisiones.

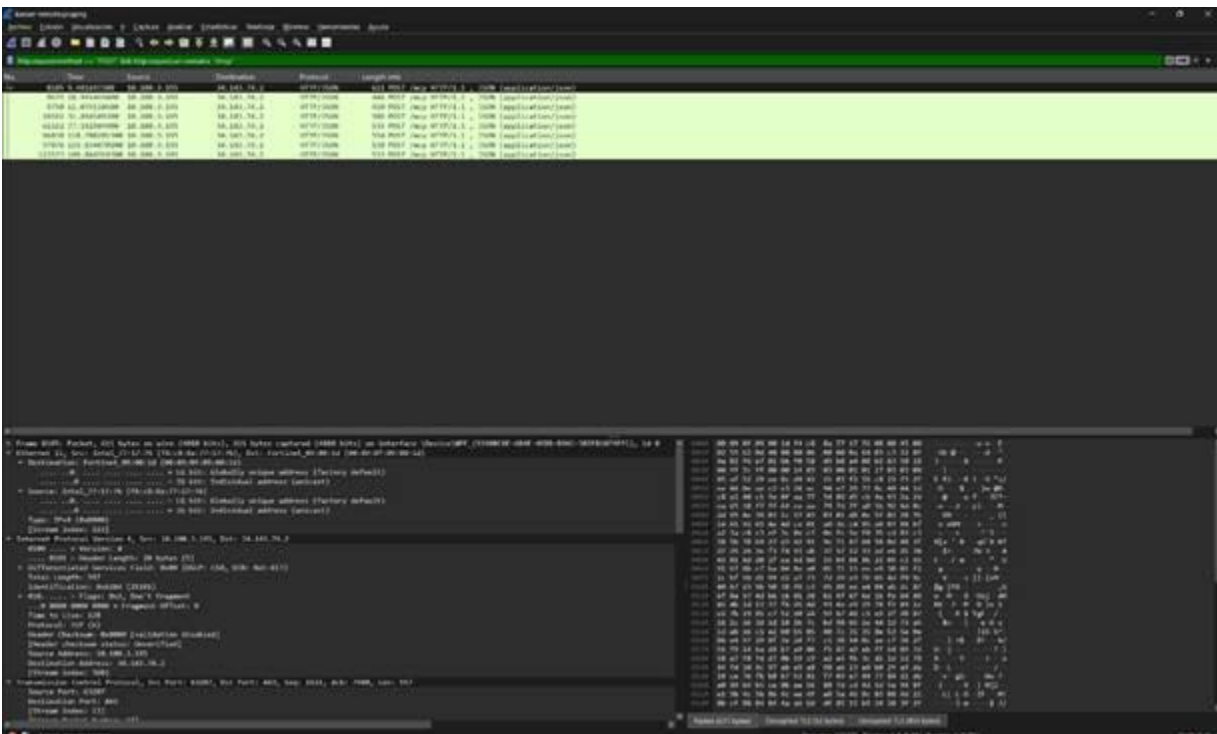


Figura 4. Detalle de direccionamiento IPv4 y segmentos TCP en el flujo cliente-servidor.

8.4 Capa de aplicación y seguridad TLS

DNS resolvió el nombre del servicio hacia infraestructura de Google. El Client Hello TLS 1.3 de la trama 7764 incluyó el nombre del servicio mediante SNI. Las tramas 8100 y 8101 contienen la respuesta del servidor, certificado, verificación y finalización; el cliente terminó el handshake en la trama 8105. TLS aportó confidencialidad en tránsito y autenticación del frontend de Cloud Run.

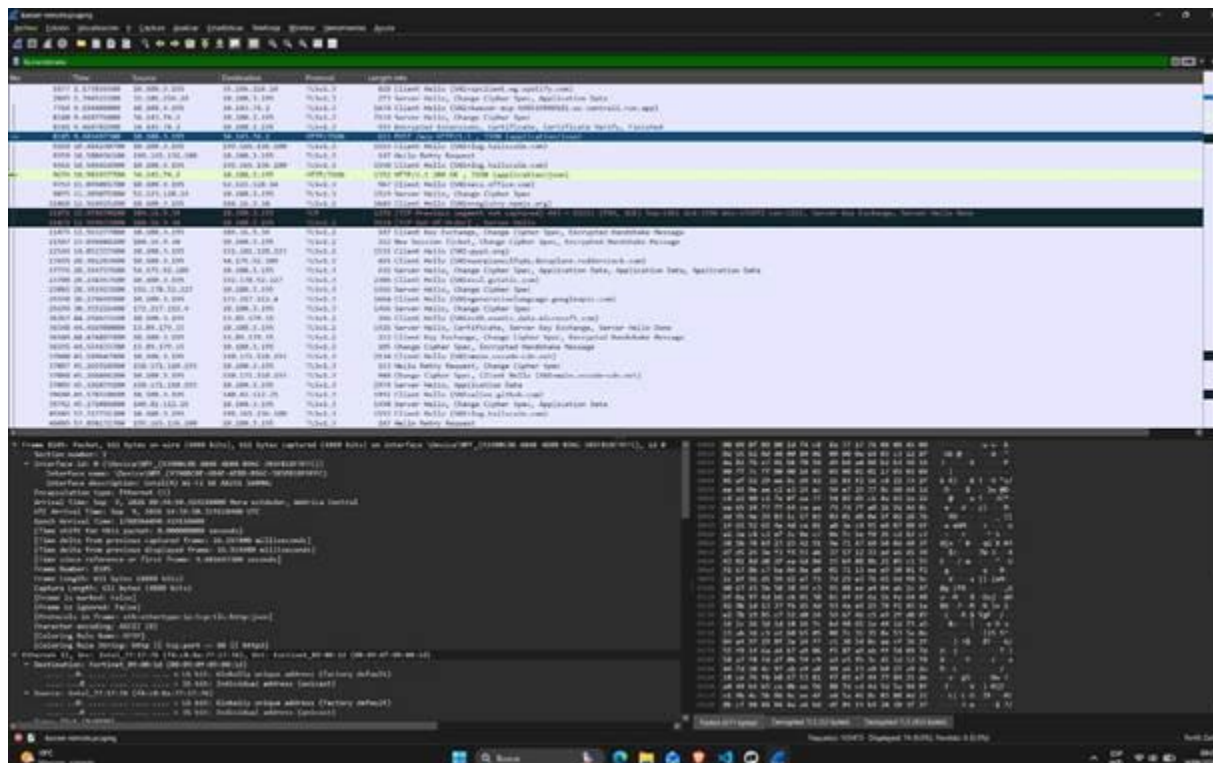


Figura 5. Negociación TLS 1.3 previa a las solicitudes HTTP del protocolo MCP.

8.5 Sincronización, solicitudes y respuestas

initialize negocia versión e identidad. La respuesta conserva el id de la solicitud. notifications/initialized no incluye id y, por tanto, no genera una respuesta JSON-RPC; el transporte acusa recepción con HTTP 202. Después, tools/list descubre capacidades y tools/call ejecuta operaciones.

Solicitud	Tiempo	Mensaje	Clase	Respuesta
8105	9.481697 s	initialize · id 1	Sincronización	9674 · HTTP 200 · id 1
9675	10.991469 s	notifications/initialized	Notificación	9749 · HTTP 202 · sin cuerpo
9750	11.079111 s	tools/list · id 2	Descubrimiento	9880 · HTTP 200 · 5 tools
26592	31.494549 s	consultar_specs · id 3	Operación	26654 · HTTP 200
62162	77.142945 s	diagnosticar_falla · id 4	Operación	62232 · HTTP 200
96838	118.708201 s	mantenimiento · id 5	Operación	96894 · HTTP 200
97876	119.834070 s	repuesto · id 6	Operación	97956 · HTTP 200 · isError
123573	149.864769 s	repuesto · id 7	Operación	123635 · HTTP 200 · isError

Un estado HTTP 200 confirma que la capa de transporte procesó el mensaje; no garantiza que la herramienta haya encontrado el dato. Las consultas de repuestos desconocidos devolvieron HTTP 200 con isError en el resultado MCP. Los identificadores y contenidos coinciden con el log JSONL del chatbot.

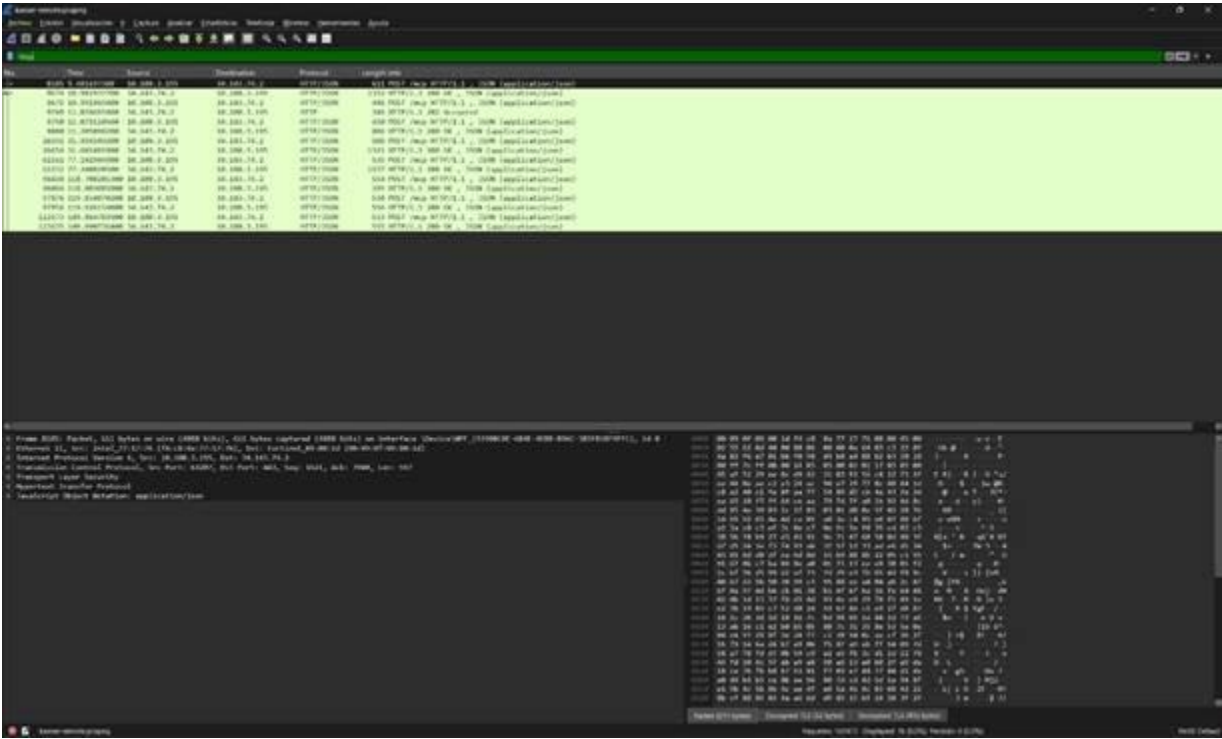


Figura 6. Pares POST /mcp y respuestas HTTP observados tras descifrar la sesión TLS propia.

9. Pruebas y validación

Las pruebas automatizadas no requieren credenciales reales. Cubren lógica industrial, persistencia, framing stdio, JSON-RPC, seguridad HTTP, clientes MCP, adaptadores de Gemini, Groq y Claude, ejecución de herramientas, contexto e identificadores de función.

Validación	Resultado	Cobertura principal
mcp-kaeser-server npm test	6/6 aprobadas	Herramientas, validación, persistencia, stdio y HTTP.
chatbot npm test	11/11 aprobadas	LLMs, MCP local/remoto, herramientas, contexto y reintentos.
check:mcp	31 herramientas	5 KAESER remotas, 14 Filesystem y 12 Git.
Demostración LLM	Correcta	Gemini llamó herramientas remotas y redactó respuestas.
Filesystem + Git	Correcta	Archivo, staging, commit y log en repositorio aislado.

Wireshark	Correcta	Flujo DNS/TCP/TLS/HTTP/JSON-RPC correlacionado.
-----------	----------	---

10. Seguridad y limitaciones

10.1 Controles implementados

- Credenciales solo mediante variables de entorno; archivos .env, config.json, logs, capturas y secretos TLS están ignorados.
- Autenticación Bearer para POST /mcp y validación de encabezados, versión, origen y tamaño de solicitud.
- Contenedor sin privilegios y separación estricta entre stdout de protocolo y stderr de diagnóstico.
- Filesystem y Git restringidos a un repositorio de demostración, fuera del repositorio principal.
- Advertencias industriales en todas las respuestas; no existe función de control físico.

10.2 Limitaciones

- Los datos operativos, inventario y mantenimiento son simulados.
- La persistencia de Cloud Run en /tmp es efímera.
- El prototipo usa Bearer y no implementa OAuth.
- El transporte remoto es stateless y no ofrece notificaciones servidor-cliente por SSE.
- La disponibilidad y cuota del LLM dependen del proveedor externo.

11. Dificultades y soluciones

Dificultad	Solución aplicada	Aprendizaje
Reutilizar lógica por stdio y HTTP	Responder inyectable y núcleo JSON-RPC común.	Separar transporte de semántica.
Mensajes parciales por stdin	Buffer delimitado por nuevas líneas.	El framing debe ser explícito.
Nombres repetidos entre servidores	Prefijo por servidor en el catálogo del LLM.	El host debe mantener enrutamiento estable.
Auditoría sin filtrar secretos	Log JSONL de mensajes MCP, sin encabezados de autenticación.	Observabilidad y confidencialidad deben coexistir.
Contenido HTTPS cifrado	Secretos TLS temporales de una sesión propia.	Wireshark necesita claves de sesión para inspeccionar HTTP.

Diferencias entre APIs LLM	Adaptadores a un formato interno común.	La integración no debe acoplar MCP a un proveedor.
Cuotas, demanda y timeouts	Modelo gratuito estable, reintentos exponenciales y proveedor alternativo.	Las fallas externas deben tratarse como transitorias.

12. Conclusiones

MCP separa el razonamiento del modelo de las acciones y fuentes de datos. El mismo catálogo KAESER pudo exponerse local y remotamente porque la semántica JSON-RPC se mantuvo independiente del transporte. La negociación inicializa y el descubrimiento tools/list permitieron integrar servidores sin codificar cada herramienta directamente en el chatbot.

La implementación manual evidenció que un protocolo real exige framing, correlación de identificadores, notificaciones, validación, clasificación de errores y seguridad. El despliegue remoto añadió DNS, TCP, TLS, HTTP, autenticación y decisiones de persistencia que no aparecen en stdio.

Los logs del anfitrión y Wireshark resultaron complementarios, el log conserva la semántica completa de JSON-RPC, mientras la captura demuestra cómo esos mensajes viajan por las capas de la red. Las pruebas, el escenario Filesystem+Git y la ejecución con Gemini confirman el funcionamiento integral requerido por el proyecto.

Referencias

1. JSON-RPC 2.0 specification. (s. f.). <https://www.jsonrpc.org/specification>
2. Model Context Protocol. (2026, 8 septiembre). Architecture overview. <https://modelcontextprotocol.io/docs/2026-07-28/learn/architecture>
3. Model Context Protocol. (2026b, septiembre 8). Overview. <https://modelcontextprotocol.io/specification/2026-07-28/basic/transport>
4. Generating content | Gemini API | Google AI for Developers. (s. f.). Google AI For Developers. <https://ai.google.dev/api/generate-content>
5. Function calling with the Gemini API. (s. f.). Google AI For Developers. <https://ai.google.dev/gemini-api/docs/function-calling>
6. Modelcontextprotocol. (s. f.). GitHub - modelcontextprotocol/servers: Model Context Protocol Servers. GitHub. <https://github.com/modelcontextprotocol/servers>

7. Contrato de entorno de ejecución del contenedor. (s. f.). Google Cloud Documentation. <https://docs.cloud.google.com/run/docs/container-contract?hl=es-419>
8. KAESER SM Series brochure, sección de especificaciones técnicas.
9. SIGMA CONTROL 2 SCREW FLUID $\geq 4.5.X$ User Manual, sección 10.2, p. 197.
10. Datos simulados del proyecto: mantenimiento.json, inventario.json y rangos-sensores.json.